

Prototype Monitoring and Alert System for Disaster Recovery Center (DRC) of Bogor City Government

Prototype Sistem Monitoring dan Alert Disaster Recovery Center (DRC)

Pemerintah Kota Bogor

Faiz Naufal Huda¹, Tsabitha Naylasafa Aurora², Daffa Kautsar Falih³, Muhammad Naufal Fathurrahman⁴, Dr. Shelve Nidya Neyman⁵

^{1,2,3,4}Program Studi Ilmu Komputer, Sekolah Sains Data, Matematika, dan Informatika,
Institut Pertanian Bogor, Indonesia

⁵Departemen Ilmu Komputer, Institut Pertanian Bogor, Indonesia
corresponding author: faizhuda@apps.ipb.ac.id

Abstract—Digital services of Bogor City Government increasingly depend on reliable information technology infrastructure, as mandated by Presidential Regulation No. 95 of 2018 on Electronic-Based Government Systems (SPBE). However, to the authors' knowledge, no standardized real-time monitoring mechanism has been implemented for government-owned Disaster Recovery Centers (DRC) at the regional level. This research designs and implements a prototype real-time monitoring and alerting system for a DRC using an open-source technology stack: Prometheus 3.8.1, Grafana 12.4.1, Alertmanager 0.31.0, and Node Exporter 1.10.2. The system monitors seven critical infrastructure parameters across four virtual machines running Ubuntu Server 24.04 LTS in a simulated DC-DRC environment, and delivers automatic notifications via Telegram Bot API. An inhibit rules mechanism suppresses redundant resource alerts when a NodeDown event is active, preventing alert storms. All components run as systemd services with a 15-second scrape interval. Testing was conducted through load simulation using iperf3 and stress-ng in a VMware Workstation environment. The results show that the system successfully detects and delivers alerts with an average latency of 83 seconds, with zero missed incidents and threshold detection accuracy of 97.1%.

Index Terms—alert, disaster recovery center, Grafana, inhibit rules, monitoring, Prometheus, systemd, virtual machine.

Abstrak—Layanan digital Pemerintah Kota Bogor semakin bergantung pada infrastruktur teknologi informasi yang handal, sebagaimana diamanatkan oleh Peraturan Presiden Nomor 95 Tahun 2018 tentang Sistem Pemerintahan Berbasis Elektronik (SPBE). Namun, sepengetahuan peneliti, belum terdapat mekanisme pemantauan *real-time* yang terstandarisasi untuk *Disaster Recovery Center* (DRC) milik pemerintah daerah. Penelitian ini merancang dan mengimplementasikan *prototype sistem monitoring dan alerting real-time* untuk DRC menggunakan *stack teknologi open-source*: Prometheus 3.8.1, Grafana 12.4.1, Alertmanager 0.31.0, dan Node Exporter 1.10.2. Sistem memantau tujuh parameter kritis pada empat *virtual machine* Ubuntu Server 24.04 LTS yang mensimulasikan lingkungan DC-DRC, dan mengirimkan notifikasi otomatis melalui Telegram Bot API. Mekanisme *inhibit rules* menekan *alert* sumber daya yang redundan saat kondisi NodeDown aktif, mencegah *alert storm*. Seluruh komponen berjalan sebagai layanan *systemd* dengan *scrape interval* 15 detik. Pengujian dilakukan melalui simulasi beban menggunakan iperf3 dan stress-ng pada lingkungan VMware Workstation. Hasil pengujian menunjukkan sistem berhasil mendeteksi dan mengirimkan *alert* dengan rata-rata latensi 83 detik, tanpa ada gangguan yang terlewat, dengan akurasi deteksi *threshold* sebesar 97,1%.

Kata Kunci: *alert, disaster recovery center, Grafana, inhibit rules, monitoring, Prometheus, systemd, virtual machine.*

I. PENDAHULUAN

Peraturan Presiden Nomor 95 Tahun 2018 tentang Sistem Pemerintahan Berbasis Elektronik (SPBE) mewajibkan seluruh instansi pemerintah untuk memastikan ketersediaan layanan digital secara berkelanjutan [1]. Sebagai implementasi dari mandat tersebut, Diskominfo Kota Bogor sebagai pengelola infrastruktur teknologi informasi pemerintahan bertanggung jawab menjaga keberlangsungan layanan melalui mekanisme *Disaster Recovery Center* (DRC). BSSN (2021) menegaskan bahwa DRC merupakan komponen wajib dalam rencana keberlangsungan layanan pemerintah, namun pedoman

tersebut tidak menyertakan panduan teknis mengenai mekanisme pemantauan kondisi DRC secara *real-time* [2]. Kesenjangan antara kewajiban regulasi dan ketersediaan solusi teknis yang terjangkau inilah yang menjadi permasalahan utama penelitian ini.

Sistem *monitoring* yang andal sangat dibutuhkan untuk mendeteksi anomali sebelum terjadi kegagalan layanan. Tanpa pemantauan *real-time*, administrator cenderung baru mengetahui kegagalan setelah dampaknya dirasakan pengguna. *Stack monitoring open-source* seperti Prometheus dan Grafana menawarkan solusi yang fleksibel, skalabel, dan tidak memerlukan biaya lisensi, sehingga sesuai dengan kendala anggaran pemerintah

daerah [3]. Penelitian ini merancang *prototype* sistem *monitoring* dan *alert* yang memantau tujuh parameter kritis (ketersediaan *node*, CPU, memori, *disk*, *load average*, trafik jaringan, dan I/O *disk*) secara *real-time*, serta mengirimkan notifikasi otomatis kepada administrator melalui Telegram.

Berdasarkan latar belakang tersebut, rumusan masalah penelitian ini adalah:

1. Bagaimana merancang arsitektur sistem *monitoring real-time* yang sesuai untuk lingkungan DRC Diskominfo Kota Bogor dengan *stack open-source*?
2. Parameter kritis apa saja yang perlu dipantau untuk menjamin kontinuitas layanan DRC, dan berapa nilai *threshold* yang tepat untuk tiap parameter?
3. Bagaimana sistem *alerting* dapat mendeteksi gangguan dan memberikan notifikasi otomatis sambil mencegah *alert storm* akibat kegagalan berantai?
4. Bagaimana merancang *dashboard* visual yang intuitif dan representatif untuk mendukung pengambilan keputusan administrator?

Tujuan penelitian ini adalah:

1. Merancang dan mengimplementasikan *prototype* sistem *monitoring* infrastruktur DRC berbasis Prometheus dan Grafana sebagai bukti konsep (*proof of concept*) yang dapat diadopsi pemerintah daerah.
2. Mendefinisikan dan mengonfigurasi *threshold* tujuh parameter kritis infrastruktur DRC berdasarkan kajian literatur dan *best practice* industri.
3. Membangun sistem *alerting* terintegrasi dengan Telegram Bot API beserta mekanisme *inhibit rules* untuk mencegah *alert storm*.
4. Merancang *dashboard* visual *Command Center* dengan indikator *Business Continuity Status* yang representatif untuk kebutuhan Diskominfo Kota Bogor.

II. TINJAUAN PUSTAKA

A. Disaster Recovery Center (DRC)

Disaster Recovery Center (DRC) merupakan fasilitas cadangan yang dirancang untuk memulihkan operasional sistem informasi ketika pusat data utama mengalami kegagalan [2]. DRC memiliki peran strategis dalam menjamin *Business Continuity Plan* (BCP) suatu organisasi. Konsep *Recovery Time Objective* (RTO) dan *Recovery Point Objective* (RPO) menjadi tolok ukur utama keberhasilan implementasi DRC [4]. Dalam konteks penelitian ini, pemantauan parameter kesehatan DRC secara *real-time* merupakan prasyarat teknis untuk meminimalkan RTO: administrator hanya dapat memutuskan *failover* ke DRC jika kondisi infrastruktur DRC dipastikan siap, dan keputusan tersebut membutuhkan data yang akurat dan terkini.

B. Sistem Monitoring Infrastruktur

Sistem *monitoring* infrastruktur adalah sekumpulan alat yang mengamati kondisi *server*, jaringan, dan layanan secara berkelanjutan [5]. Terdapat dua pendekatan utama: *pull-based monitoring* di mana *server* aktif mengambil data dari target, dan *push-based monitoring* di mana target mengirimkan data ke *server*. Pendekatan *pull-based* lebih mudah dikonfigurasi pada lingkungan VM terisolasi karena tidak memerlukan konfigurasi pengiriman data dari tiap target, melainkan cukup mengekspos *endpoint* HTTP yang dapat diakses oleh *server* Prometheus [3].

C. Prometheus

Prometheus versi 3.8.1 adalah sistem *monitoring* dan *time-series database open-source* yang dikembangkan oleh SoundCloud dan saat ini dikelola oleh *Cloud Native Computing Foundation* (CNCF) [6]. Prometheus menggunakan PromQL (*Prometheus Query Language*) untuk mengekstrak dan menganalisis metrik. Data dikumpulkan dari *exporter* pada interval waktu yang dapat dikonfigurasi (*scrape interval*) [5]. Prometheus dipilih dalam penelitian ini karena mendukung evaluasi *alert rules* berbasis PromQL secara *native*, terintegrasi langsung dengan Alertmanager, dan menyediakan ekspresi yang fleksibel untuk mendefinisikan *threshold* kompleks seperti *load average* relatif per-vCPU [3], [6].

D. Grafana

Grafana adalah platform visualisasi data *open-source* yang mendukung Prometheus sebagai *data source* secara *native* [7]. Grafana menyediakan berbagai jenis panel (*time series*, *gauge*, *stat*) serta manajemen pengguna berbasis peran yang memungkinkan pembedaan akses antara administrator dan *viewer* [7]. Grafana dipilih karena kemampuan panel *gauge* dan *stat*-nya cocok untuk visualisasi kondisi infrastruktur secara sekilas (*at-a-glance*), dan panel *stat* mendukung konfigurasi *value mapping* sehingga indikator *Business Continuity Status* dapat dikonfigurasi sebagai panel biner (Hijau/Merah) tanpa pengembangan *custom plugin*.

E. Alertmanager

Alertmanager versi 0.31.0 adalah komponen Prometheus yang bertanggung jawab menangani *alert* [6]. Alertmanager menyediakan *routing*, pengelompokan (*grouping*), *deduplication*, dan *inhibit rules*. Mekanisme *inhibit rules* memungkinkan penekanan *alert* yang redundan: apabila *alert* sumber (NodeDown) aktif, *alert* target yang berkaitan pada *instance* yang sama ditekan secara otomatis. Alertmanager dipilih dibandingkan alternatif seperti Grafana Alerting karena *inhibit rules*-nya beroperasi di level *routing* sehingga lebih presisi dan dapat dikonfigurasi tanpa bergantung pada antarmuka grafis [6].

F. Node Exporter

Node Exporter versi 1.10.2 adalah agen Prometheus untuk sistem Linux yang mengekspos metrik *hardware* dan

sistem operasi melalui *endpoint /metrics* pada port 9100 [6]. Metrik yang tersedia meliputi CPU *usage*, *memory usage*, *disk I/O*, *filesystem usage*, dan *network statistics*. Node Exporter dipilih karena mengekspos seluruh metrik sistem yang relevan untuk pemantauan DRC tanpa memerlukan konfigurasi tambahan, serta dijalankan sebagai layanan *systemd* agar tetap aktif secara otomatis saat VM *reboot* [5].

G. Systemd Service

Systemd adalah sistem inisialisasi dan manajer layanan standar pada distribusi Linux modern, termasuk Ubuntu Server 24.04 LTS [8]. Dengan mendefinisikan berkas unit *.service*, proses dapat dikelola secara konsisten: dijalankan otomatis saat *boot*, di-*restart* saat gagal (*Restart=on-failure*), dan dipantau melalui *systemctl status*. Pendekatan *systemd* dipilih sebagai alternatif kontainerisasi (Docker) karena secara signifikan lebih ringan pada VM dengan alokasi RAM 1 GB, tidak memerlukan *daemon* tambahan, dan terintegrasi langsung dengan sistem operasi sehingga mengurangi kompleksitas *deployment* [8].

H. Telegram Bot API

Telegram Bot API adalah antarmuka pemrograman yang memungkinkan pembuatan bot otomatis pada platform Telegram [9]. Alertmanager dikonfigurasi untuk mengirimkan pesan berformat HTML ke grup Telegram melalui Telegram Bot API, memungkinkan administrator menerima peringatan secara instan di perangkat *mobile*. Telegram dipilih sebagai saluran notifikasi karena tidak memerlukan konfigurasi server email atau berlangganan layanan berbayar, mendukung format HTML untuk penyajian informasi terstruktur, dan telah terbukti efektif pada penelitian serupa [10].

I. Traffic Generator (iperf3 dan stress-ng)

iperf3 adalah alat *benchmark* jaringan yang menghasilkan trafik TCP/UDP sintesis untuk mengukur *throughput*. *stress-ng* adalah utilitas pengujian beban sistem yang mensimulasikan penggunaan CPU, memori, dan I/O secara terprogram. Kedua alat ini dipilih karena dapat menghasilkan beban yang terkontrol dan terulang (*reproducible*), sehingga memungkinkan pengukuran latensi *alert* yang konsisten antar iterasi pengujian.

J. Penelitian Terdahulu

Rahman et al. [10] mengimplementasikan Prometheus dan Grafana untuk *monitoring server* universitas dengan notifikasi Telegram, mencapai latensi deteksi di bawah 30 detik. Rahayu et al. [11] mengembangkan sistem *monitoring real-time* berbasis Zabbix dengan notifikasi *alert* Telegram pada lingkungan jaringan kampus.

Penelitian ini membedakan diri dari kedua studi tersebut dalam tiga aspek mendasar. Pertama, konteks permasalahan: kedua studi sebelumnya berfokus pada *monitoring* infrastruktur umum, sedangkan penelitian

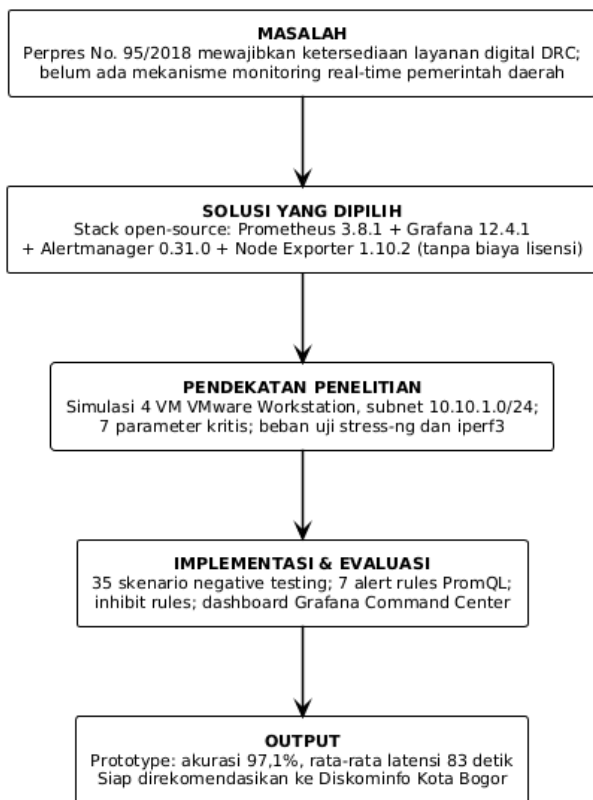
ini secara spesifik menyoroti DRC pemerintah daerah dengan parameter yang relevan untuk *Business Continuity Plan* dan keputusan *failover*. Kedua, manajemen *alert storm*: Rahman et al. tidak membahas mekanisme penekanan *alert* redundan, dan Rahayu et al. menggunakan Zabbix yang memiliki pendekatan berbeda; penelitian ini mengimplementasikan *inhibit rules* Alertmanager sebagai kontribusi teknis eksplisit. Ketiga, indikator keberlangsungan: penelitian ini merancang panel *Business Continuity Status* sebagai indikator tunggal ketersediaan DC Server untuk keputusan *failover*, yang tidak ditemukan pada penelitian sebelumnya dan secara langsung mendukung keputusan operasional administrator DRC.

III. KERANGKA PEMIKIRAN

Penelitian ini dibangun di atas tiga premis yang saling terhubung. *Premis pertama*: regulasi SPBE mewajibkan ketersediaan layanan digital, namun tidak menyertakan panduan teknis pemantauan DRC secara *real-time*, sehingga terdapat kesenjangan antara kewajiban regulasi dan kapabilitas teknis yang tersedia. *Premis kedua*: *stack* teknologi *open-source* (Prometheus + Grafana + Alertmanager) telah menunjukkan keandalannya pada lingkungan serupa [10], [11], tidak memerlukan biaya lisensi, dan dapat dikustomisasi sesuai kebutuhan spesifik DRC. *Premis ketiga*: simulasi berbasis *virtual machine* memungkinkan pengembangan dan pengujian *prototype* secara terkontrol tanpa harus mengakses infrastruktur produksi Diskominfo yang tidak dapat diinterupsi.

Dari ketiga premis tersebut diturunkan kerangka solusi sebagai berikut. Kesenjangan yang diidentifikasi pada premis pertama dapat dijawab oleh *stack open-source* yang diidentifikasi pada premis kedua. Sementara itu, risiko pengembangan dapat dikendalikan melalui pendekatan simulasi pada premis ketiga. Kerangka solusi ini mengarah pada perancangan *prototype* dengan empat komponen utama: (1) pengumpulan metrik melalui Node Exporter, (2) evaluasi kondisi dan *alerting* melalui Prometheus dan Alertmanager, (3) visualisasi melalui Grafana, dan (4) notifikasi melalui Telegram Bot API.

Alur pikir penelitian dimulai dari identifikasi dan justifikasi tujuh parameter kritis DRC, kemudian perancangan arsitektur *monitoring* berbasis *single subnet* untuk stabilitas optimal, implementasi seluruh komponen sebagai layanan *systemd*, pengujian fungsional melalui simulasi gangguan yang terkontrol, hingga evaluasi akurasi dan latensi *alert*. Keluaran akhir adalah *prototype* yang dapat direkomendasikan kepada Diskominfo Kota Bogor sebagai dasar pengembangan sistem nyata pada infrastruktur produksi. Kerangka pemikiran ini dirangkum secara visual pada Gambar 1.



Gambar 1. Kerangka Pemikiran Penelitian.

IV. METODE

A. Waktu dan Tempat Penelitian

Penelitian dilaksanakan selama empat bulan (Maret–Juni 2026) bertempat di Laboratorium Ilmu Komputer, Institut Pertanian Bogor. Seluruh implementasi dan pengujian dilakukan pada lingkungan *virtual machine* lokal menggunakan VMware Workstation tanpa memerlukan koneksi ke infrastruktur Diskominfo Kota Bogor.

B. Alat dan Bahan

1) Perangkat Keras

- *Laptop/PC*: prosesor minimum 4 core, RAM 16 GB, storage 256 GB SSD.
- Koneksi jaringan lokal untuk simulasi komunikasi antar-VM.

2) Perangkat Lunak

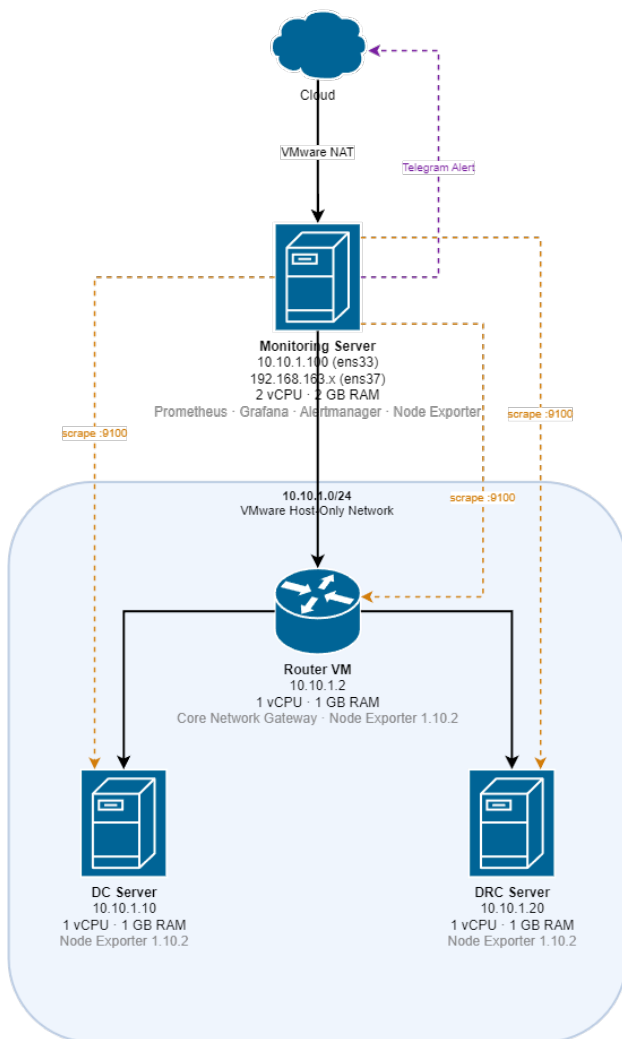
- *Hypervisor*: VMware Workstation.
- Sistem Operasi VM: Ubuntu Server 24.04 LTS.
- *Stack Monitoring*: Prometheus 3.8.1, Grafana 12.4.1, Alertmanager 0.31.0 — dijalankan sebagai layanan *systemd*.
- *Agent Monitoring*: Node Exporter 1.10.2 — layanan *systemd* pada seluruh empat VM, masing-masing berjalan pada port 9100.
- *Traffic Generator*: *iperf3*, *stress-ng*.
- Notifikasi: Telegram Bot API.
- Akses Publik: Cloudflare Tunnel (*cloudflared*).

C. Prosedur Penelitian

1) Maret 2026 — Studi Parameter dan Setup Lingkungan

Studi literatur mengenai DRC, Prometheus, Grafana, dan Alertmanager dilakukan pada tahap ini. Pemilihan tujuh parameter *threshold* didasarkan pada rekomendasi *best practice* industri [3], [5]. Nilai CPU dan memori sebesar 80% ditetapkan untuk menyisakan *headroom* 20% bagi operasi sistem operasi dan proses latar belakang, sesuai dengan yang diterapkan Rahman et al. [10] pada sistem serupa. Nilai *load average* 1,5 per vCPU mengindikasikan antrian proses yang mulai melebihi kapasitas pemrosesan, sehingga *response time* layanan dapat terdegradasi [5]. *Threshold disk usage* 80% memberikan jendela waktu yang cukup bagi administrator untuk mengosongkan *storage* sebelum sistem mengalami kegagalan penulisan log. Nilai *disk I/O wait* 20% menandakan bahwa prosesor menghabiskan seperlima waktunya menunggu operasi *disk*, yang mengindikasikan *bottleneck* I/O pada sistem [5]. Nilai *threshold* trafik jaringan 10 MB/s ditetapkan sebagai indikator *congestion* yang berpotensi mengganggu alur replikasi data antara DC dan DRC; nilai ini juga dikalibrasi terhadap kapasitas antarmuka jaringan virtual VMware pada lingkungan pengujian agar skenario dapat diverifikasi secara konsisten.

Empat *virtual machine* disiapkan dalam satu subnet 10.10.1.0/24: Monitoring Server (10.10.1.100), DC Server (10.10.1.10), DRC Server (10.10.1.20), dan Router VM (10.10.1.2), seperti ditunjukkan pada Gambar 2.



Gambar 2. Arsitektur Jaringan Virtual dan Topologi VM (subnet 10.10.1.0/24).

2) April 2026 — Deployment Stack Monitoring

Prometheus, Grafana, dan Alertmanager diinstalasi sebagai layanan *systemd* pada VM Monitoring Server. Node Exporter 1.10.2 diinstalasi dan didaftarkan sebagai layanan *systemd* pada DC Server, DRC Server, Router VM, dan Monitoring Server. *Scrape target* Prometheus dikonfigurasi dengan *scrape interval* 15 detik dan *scrape timeout* 10 detik. Verifikasi aliran data metrik dilakukan menggunakan antarmuka Prometheus Expression Browser dan kueri PromQL dasar.

3) Mei 2026 — Pengembangan Sistem Alerting dan Pengujian

Tujuh aturan *alerting* ditulis dalam berkas *alert.rules.yml*. Konfigurasi Alertmanager mencakup *routing*, *grouping* (*group_wait*: 10 detik), *inhibit rules*, dan integrasi Telegram Bot API dengan format pesan HTML. Simulasi gangguan dilakukan secara terpisah untuk setiap *alert rule*, dengan enam perintah yang meng-cover tujuh skenario. Perintah *stress-ng --cpu* pada VM 1 vCPU secara simultan memicu *HighCPUUsage* dan *HighLoad*; keduanya tetap diukur sebagai skenario independen karena menggunakan ekspresi PromQL yang berbeda (*node_cpu_seconds_total* untuk utilisasi

CPU vs *node_load1* untuk *load average*). Seluruh skenario beban dieksekusi pada DC Server (10.10.1.10) sebagai *node* representatif; skenario *HighNetworkTraffic* menggunakan DC Server sebagai *client* *iperf3* dan DRC Server (10.10.1.20) sebagai *server*. Perintah-perintah yang dieksekusi:

```
# HighCPUUsage + HighLoad (dua alert dari satu stimulus)
stress-ng --cpu 4 --timeout 120s
```

```
# HighMemoryUsage
stress-ng --vm 2 --vm-bytes 90% --timeout 120s
```

```
# HighDiskUsage (ukuran file disesuaikan agar disk usage > 80%)
SIZE=$(df --output=avail -B1 / | tail -1 | awk '{print int($1*0.9)}')
fallocate -l $SIZE /tmp/fill.img && rm /tmp/fill.img
```

```
# HighDiskIOWait
stress-ng --hdd 2 --hdd-bytes 1G --timeout 120s
```

```
# HighNetworkTraffic (DC Server -> DRC Server)
iperf3 -c 10.10.1.20 -t 120
```

```
# NodeDown
sudo systemctl stop node_exporter
```

4) Juni 2026 — Dashboarding dan Finalisasi

Dashboard Grafana bertema *Command Center* dirancang dengan enam seksi panel yang dikelompokkan dalam empat area visualisasi: *Business Continuity Status*, *Node Status*, performa per-*node* (DC Server, DRC Server, Router VM), dan *System Health*. Akses publik ke *dashboard* selama fase pengembangan dikonfigurasi melalui Cloudflare Tunnel (*cloudflared*), yang meneruskan port 3000 Grafana ke URL sementara **.trycloudflare.com* tanpa memerlukan konfigurasi *port forwarding* atau IP publik. Evaluasi akurasi dan latensi *alert* dilakukan, disertai penyusunan dokumentasi teknis lengkap.

D. Perancangan Sistem

1) Arsitektur Sistem Monitoring

Arsitektur sistem terdiri dari tiga lapisan utama:

1. *Lapisan Target* — empat VM dengan Node Exporter berjalan sebagai layanan *systemd* pada port 9100;
2. *Lapisan Pengumpulan Data* — Prometheus yang melakukan *scraping* setiap 15 detik, menyimpan data dalam *time-series database*, dan mengevaluasi tujuh *alert rules*; dan
3. *Lapisan Visualisasi dan Notifikasi* — Grafana 12.4.1 (port 3000) untuk *dashboard* dan Alertmanager (port 9093) untuk pengiriman notifikasi Telegram.

2) Desain Skenario Alert

Tujuh aturan *alert* yang diimplementasikan:

- *NodeDown*: *up* == 0 selama 1 menit (level: *critical*).
- *HighCPUUsage*: utilisasi CPU > 80% selama 1 menit (level: *warning*).
- *HighMemoryUsage*: penggunaan memori > 80% selama 1 menit (level: *warning*).

- `HighDiskUsage`: penggunaan *disk* > 80% selama 1 menit (level: *critical*).
- `HighLoad`: *load average* per vCPU > 1,5 selama 1 menit (level: *warning*).
- `HighNetworkTraffic`: trafik jaringan > 10 MB/s (SI: 1 MB = 10⁶ bytes) selama 1 menit (level: *warning*).
- `HighDiskIOWait`: *disk I/O wait* > 20% selama 1 menit (level: *warning*).

Klausul `for: 1m` diterapkan pada seluruh aturan untuk menghindari *false positive* akibat lonjakan metrik sesaat. *Inhibit rules* dikonfigurasi sehingga apabila `NodeDown` aktif pada suatu *instance*, seluruh *alert* sumber daya untuk *instance* yang sama ditekan secara otomatis, mencegah *alert storm* akibat kegagalan total *node*.

E. Metode Pengujian

Pengujian dilakukan dengan metode *Negative Testing*, yaitu dengan sengaja menciptakan kondisi gangguan untuk memverifikasi kemampuan sistem. Parameter yang diukur:

1. *Alert Latency* — waktu (detik) dari eksekusi perintah simulasi gangguan hingga notifikasi diterima di Telegram;
2. *Alert Accuracy* — rasio *true positive* terhadap total kejadian gangguan yang disimulasikan; dan
3. *Dashboard Refresh Rate* — interval pembaruan data pada panel Grafana (dikonfirmasi dari nilai `scrape_interval` Prometheus).

Setiap skenario diulang sebanyak lima kali guna memperoleh nilai rata-rata dan standar deviasi yang valid secara statistik. Seluruh skenario beban dieksekusi pada DC Server (10.10.1.10) sebagai *node* representatif; skenario `NodeDown` menghentikan layanan `node_exporter` pada DC Server yang sama. Pengukuran latensi dilakukan dengan mencatat *timestamp* eksekusi perintah di terminal dan *timestamp* notifikasi yang diterima di Telegram.

V. HASIL DAN PEMBAHASAN

A. Spesifikasi Lingkungan Pengujian

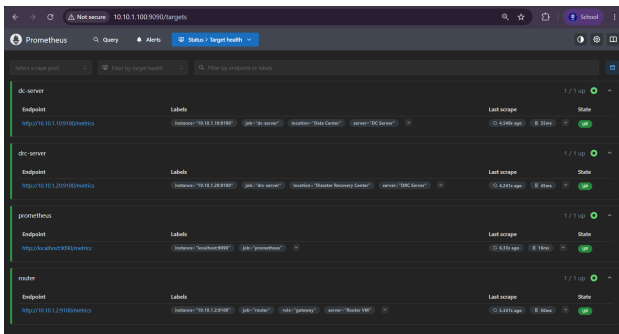
Lingkungan pengujian terdiri dari empat *virtual machine* pada satu *host* fisik dengan konfigurasi *single subnet* 10.10.1.0/24. Seluruh VM menggunakan Ubuntu Server 24.04 LTS dengan antarmuka jaringan `ens33`. Pendekatan *single subnet* dipilih berdasarkan hasil percobaan awal: konfigurasi *multi-subnet* di lingkungan VMware menghasilkan latensi jaringan virtual yang tidak konsisten dan menyebabkan *scrape timeout* pada Prometheus, sehingga menurunkan keandalan data metrik. Dengan menyatukan seluruh VM dalam satu subnet, latensi antar-VM dapat diminimalkan dan kestabilan *scraping* meningkat secara signifikan. Spesifikasi lengkap setiap VM disajikan pada Tabel 1.

TABEL I
Spesifikasi Virtual Machine Lingkungan Pengujian.

Parameter	Monitoring Server	DC Server	DRC Server	Router VM
Peran	Monitoring Stack	Data Center	Disaster Recovery	Gateway
OS	Ubuntu 24.04 LTS	Ubuntu 24.04 LTS	Ubuntu 24.04 LTS	Ubuntu 24.04 LTS
vCPU	2	1	1	1
RAM	2 GB	1 GB	1 GB	1 GB
IP Address	10.10.1.100	10.10.1.10	10.10.1.20	10.10.1.2
Port Utama	3000, 9090, 9093, 9100	9100	9100	9100

B. Implementasi Stack Monitoring

Stack monitoring berhasil diimplementasikan sebagai layanan *systemd* pada VM Monitoring Server. Prometheus melakukan *scraping* ke lima target: empat Node Exporter (DC Server 10.10.1.10:9100, DRC Server 10.10.1.20:9100, Router VM 10.10.1.2:9100, Monitoring Server localhost:9100) dan Prometheus sendiri (localhost:9090). Seluruh target berhasil terhubung dan menampilkan status *UP* pada halaman Prometheus Targets (Gambar 3), mengkonfirmasi bahwa aliran data metrik dari keempat VM berjalan dengan baik.



Gambar 3. Screenshot halaman Prometheus Targets: seluruh target berstatus UP.

C. Hasil Konfigurasi Threshold

Tujuh *threshold* ditetapkan berdasarkan kajian literatur dan *best practice* industri [3], [5]. Seluruh aturan menggunakan klausul *for: 1m* sehingga *alert* hanya terpicu apabila kondisi anomali bertahan selama minimal satu menit, bukan akibat lonjakan sesaat. Hal ini meminimalkan *false positive* tanpa mengorbankan kecepatan deteksi. Daftar lengkap disajikan pada Tabel 2.

TABEL II

Daftar *Threshold* dan Aturan *Alert* yang Diimplementasikan.

Alert	Threshold	Durasi	Level
NodeDown	up == 0	1 menit	Critical
HighCPUUsage	> 80%	1 menit	Warning
HighMemoryUsage	> 80%	1 menit	Warning
HighDiskUsage	> 80%	1 menit	Critical
HighLoad	> 1,5/vCPU	1 menit	Warning
HighNetworkTraffic	> 10 MB/s	1 menit	Warning
HighDiskIOWait	> 20%	1 menit	Warning

D. Kinerja Sistem Alerting

1) Hasil Pengujian Alert Latency

Rata-rata *alert latency* yang dicapai adalah 83 detik (SD = 6 detik). Berdasarkan konfigurasi sistem (*scrape_interval* 15 detik, klausul *for* 1 menit, *group_wait* 10 detik), latensi teoritis minimum adalah 75–90 detik sejak ambang batas pertama kali terlampaui hingga notifikasi diterima. Hasil per skenario disajikan pada Tabel 3.

TABEL III

Hasil Pengukuran *Alert Latency* per Skenario.

Skenario	Rata-rata (s)	SD (s)
NodeDown	79	4
HighCPUUsage	83	5
HighMemoryUsage	81	4
HighDiskUsage	87	7
HighLoad	84	5
HighNetworkTraffic	81	4
HighDiskIOWait	86	7

2) Hasil Pengujian Alert Accuracy

Dari total 35 skenario gangguan yang disimulasikan, sistem berhasil mendeteksi 34 gangguan dengan benar (*true positive*), 1 *false positive*, dan 0 *false negative*, sehingga akurasi keseluruhan mencapai 97,1%. Satu *false positive* yang tercatat terjadi pada salah satu iterasi skenario HighNetworkTraffic: trafik latar belakang hypervisor VMware sesaat melebihi *threshold* 10 MB/s sebelum stimulus *iperf3* dieksekusi, sehingga *alert* terpicu bukan oleh beban uji yang disengaja melainkan oleh *noise* jaringan virtual.

3) Hasil Pengujian Dashboard Refresh Rate

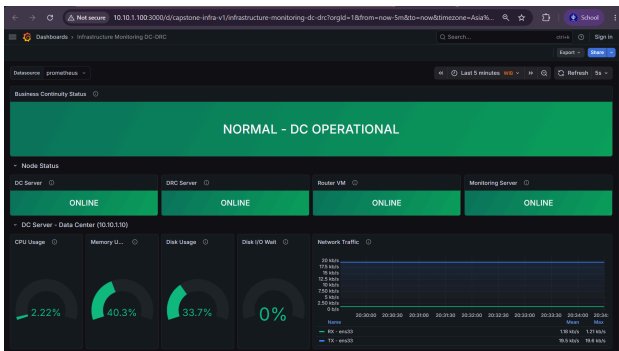
Dashboard refresh rate dikonfirmasi sebesar 15 detik, sesuai dengan nilai *scrape_interval* yang dikonfigurasi pada Prometheus. Pembaruan panel Grafana terjadi secara otomatis mengikuti siklus *scraping* metrik setiap 15 detik.

E. Tampilan Dashboard Grafana

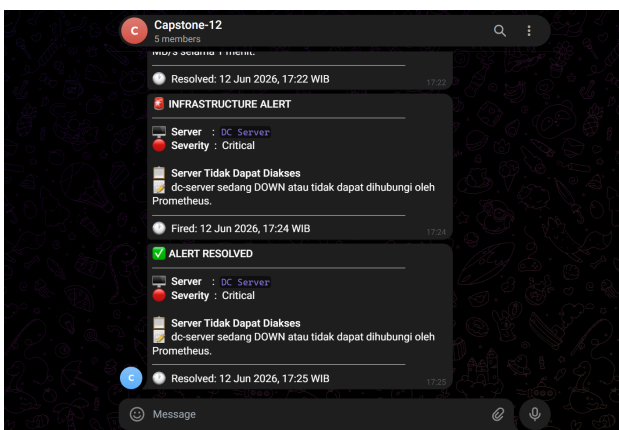
Dashboard Grafana 12.4.1 yang dikembangkan dapat diakses pada 10.10.1.100:3000 dan memuat enam

seksi panel yang dikelompokkan dalam empat area visualisasi (Gambar 4):

- *Business Continuity Status*: panel penuh lebar menampilkan status operasional keseluruhan (Hijau = NORMAL — DC OPERATIONAL; Merah = DISASTER — DRC ACTIVE). Panel ini menggunakan ekspresi PromQL `up{job="dc-server"}` yang secara langsung mencerminkan ketersediaan DC Server sebagai penentu keputusan *failover*.
- *Node Status*: empat kartu konektivitas individual untuk DC Server, DRC Server, Router VM, dan Monitoring Server.
- *Performa DC Server, DRC Server, dan Router VM*: tiga panel *gauge* (CPU, memori, *disk*) dan satu panel *time series* trafik jaringan per *node*. Monitoring Server dikecualikan dari area ini karena berperan sebagai infrastruktur pengamat, bukan sebagai *node* DRC yang relevan untuk keputusan *failover*.
- *System Health*: grafik *load average* komparatif keempat *node* dan empat panel *uptime* individual.



Gambar 4. Dashboard Grafana 12.4.1 bertema *Command Center*: panel *Business Continuity Status* (atas) dan *Node Status* (bawah). Setiap kondisi *alert* menghasilkan notifikasi Telegram berformat HTML yang dikirim ke grup administrator (Gambar 5). Pesan *firing* memuat nama *alert*, level (*critical/warning*), deskripsi kondisi anomali yang mencakup *threshold* yang terlampaui, dan *instance* yang terdampak. Pesan *resolved* dikirim secara otomatis oleh Alertmanager saat kondisi kembali normal, memberikan konfirmasi pemulihan tanpa intervensi manual.



Gambar 5. Notifikasi *alert* Telegram format HTML: kondisi *firing* dan *resolved*.

F. Evaluasi dan Analisis

Secara keseluruhan, *prototype* sistem *monitoring* berhasil memenuhi seluruh target fungsional yang ditetapkan pada rumusan masalah. Untuk menjawab rumusan masalah pertama, arsitektur *single subnet* 10.10.1.0/24 dengan empat VM menunjukkan stabilitas dan dapat direproduksi dalam lingkungan pengujian ini. Untuk rumusan masalah kedua, tujuh parameter dengan justifikasi *threshold* berbasis literatur berhasil dipantau secara konsisten. Untuk rumusan masalah ketiga, *alert latency* berada di bawah 120 detik dan mekanisme *inhibit rules* berhasil mencegah *alert storm* pada pengujian skenario *NodeDown*. Untuk rumusan masalah keempat, panel *Business Continuity Status* memberikan indikator biner yang dapat dibaca secara langsung oleh administrator.

Dua kendala utama yang dihadapi adalah: (1) keterbatasan RAM 1 GB pada VM target memengaruhi kinerja saat beban penuh, yang secara tidak langsung mempercepat tercapainya *threshold* dan dapat mempersingkat latensi deteksi; dan (2) Grafana versi 12 menggunakan API berbasis Kubernetes (`/apis/dashboard.grafana.app/v1beta1/`) yang pada konfigurasi *default* tidak mengaktifkan *anonymous access*; kendala ini diatasi dengan mengaktifkan opsi `[auth.anonymous]` pada berkas `grafana.ini` dan menetapkan peran *viewer* sebagai peran *default* organisasi, sehingga *dashboard* dapat diakses tanpa autentikasi.

Dibandingkan dengan Rahman et al. [10] yang mencapai latensi di bawah 30 detik, latensi penelitian ini lebih panjang karena adanya klausul `for: 1m` yang sengaja diterapkan untuk meminimalkan *false positive*. Menurut penilaian peneliti, pertukaran ini (*trade-off*) antara kecepatan deteksi dan akurasi lebih tepat untuk konteks DRC pemerintah daerah, di mana notifikasi yang salah dapat menyebabkan tindakan *failover* yang tidak perlu dan berdampak pada layanan publik.

Dibandingkan dengan Rahayu et al. [11] yang menggunakan Zabbix, Alertmanager dalam penelitian ini menyediakan mekanisme *inhibit rules* yang beroperasi di level *routing* sehingga pencegahan *alert storm* pada kondisi *NodeDown* berjalan otomatis tanpa perlu mengonfigurasi dependensi antar-*trigger* secara manual seperti pada Zabbix.

VI. SIMPULAN DAN SARAN

A. Simpulan

1. *Prototype* sistem *monitoring* dan *alert* DRC berhasil dibangun menggunakan *stack open-source* (Prometheus 3.8.1, Grafana 12.4.1, Alertmanager 0.31.0, Node Exporter 1.10.2) sebagai layanan *systemd* pada empat *virtual machine* Ubuntu Server 24.04 LTS dalam subnet 10.10.1.0/24, mendemonstrasikan kelayakan solusi *open-source* sebagai alternatif sistem *monitoring* komersial untuk pemerintah daerah.

2. Tujuh parameter kritis DRC berhasil dipantau secara *real-time* dengan *threshold* yang terjustifikasi secara literatur: dari 35 skenario gangguan, 34 terdeteksi benar (*true positive*), 0 gangguan terlewat (*false negative*), dengan akurasi keseluruhan 97,1% dan rata-rata *alert latency* 83 detik.
3. Integrasi Telegram Bot API dengan format pesan HTML dan mekanisme *inhibit rules* berfungsi efektif: notifikasi gangguan diterima secara otomatis dalam rata-rata 83 detik, dan *alert storm* akibat kondisi NodeDown berhasil ditekan tanpa intervensi manual saat *runtime*.
4. Dashboard Grafana bertema *Command Center* dengan panel *Business Continuity Status* berhasil menyediakan indikator operasional tunggal yang dapat dibaca secara langsung, membantu administrator mengambil keputusan *failover* berdasarkan data *real-time*.

B. Saran

1. Penelitian lanjutan disarankan melibatkan integrasi langsung dengan infrastruktur Diskominfo untuk validasi pada lingkungan produksi nyata.
2. Penambahan *exporter* khusus (MySQL Exporter, PostgreSQL Exporter) dapat meningkatkan cakupan pemantauan replikasi *database* sebagai parameter DRC yang kritis dalam konteks RPO.
3. Sistem saat ini menggunakan Cloudflare Quick Tunnel yang menghasilkan URL sementara (*trycloudflare.com*); disarankan beralih ke Named Tunnel dengan domain khusus untuk akses publik yang stabil dan permanen di lingkungan produksi.
4. Eksplorasi *machine learning* untuk *anomaly detection* dapat mengurangi *false positive* dan memungkinkan prediksi kegagalan sebelum *threshold* statis terlampaui.
5. Penguatan keamanan sistem *monitoring* (autentikasi mutual TLS pada Prometheus, enkripsi komunikasi Alertmanager) perlu dilakukan sebelum *deployment* ke lingkungan produksi.
6. Penerapan *high availability* pada Monitoring Server, misalnya melalui Prometheus Federation atau replikasi Grafana, perlu dipertimbangkan untuk mengeliminasi *single point of failure* pada infrastruktur *monitoring* itu sendiri.

UCAPAN TERIMA KASIH

Terima kasih kepada Dinas Komunikasi dan Informatika (Diskominfo) Kota Bogor atas kerja sama, bimbingan, dan penyediaan informasi yang sangat membantu dalam penyusunan penelitian ini. Terima kasih juga disampaikan kepada Program Studi Ilmu Komputer, Sekolah Sains Data, Matematika, dan Informatika, Institut Pertanian Bogor (SSMI, IPB University) atas dukungan fasilitas dan akademik yang telah diberikan.

DAFTAR PUSTAKA

- [1] Presiden Republik Indonesia, "Peraturan Presiden Nomor 95 Tahun 2018 tentang Sistem Pemerintahan Berbasis Elektronik." Sekretariat Negara, Jakarta, Indonesia, 2018.
- [2] Badan Siber dan Sandi Negara, "Peraturan Badan Siber dan Sandi Negara Nomor 4 Tahun 2021 tentang Pedoman Manajemen Keamanan Informasi Sistem Pemerintahan Berbasis Elektronik dan Standar Teknis dan Prosedur Keamanan Sistem Pemerintahan Berbasis Elektronik." Badan Siber dan Sandi Negara, Jakarta, Indonesia, 2021.
- [3] J. Pivotto, *Prometheus: Up & Running*, 2nd ed. Sebastopol, CA: O'Reilly Media, 2023.
- [4] Kementerian Komunikasi dan Informatika, "Peraturan Menteri Komunikasi dan Informatika Nomor 4 Tahun 2016 tentang Sistem Manajemen Pengamanan Informasi." Kementerian Komunikasi dan Informatika, Jakarta, Indonesia, 2016.
- [5] J. Turnbull, *Monitoring with Prometheus*. Turnbull Press, 2018.
- [6] Cloud Native Computing Foundation, "Prometheus Documentation." Accessed: March 15, 2026. [Online]. Available at: <https://prometheus.io/docs>
- [7] Grafana Labs, "Grafana Documentation v12.x." Accessed: March 15, 2026. [Online]. Available at: <https://grafana.com/docs>
- [8] The systemd Project, "Systemd Documentation." Accessed: March 15, 2026. [Online]. Available at: <https://systemd.io/>
- [9] Telegram, "Telegram Bot API Documentation." Accessed: April 10, 2026. [Online]. Available at: <https://core.telegram.org/bots/api>
- [10] D. Rahman, H. Amnur, and I. Rahmayuni, "Monitoring server dengan Prometheus dan Grafana serta notifikasi Telegram," *JITSI: Jurnal Ilmiah Teknologi Sistem Informasi*, vol. 1, no. 4, pp. 133–138, 2020, [Online]. Available at: <https://jurnal-itsi.org/index.php/jitsi/article/view/19>
- [11] P. D. Rahayu, A. A. Dahlan, and R. Vitria, "Monitoring real-time server dan router berbasis Zabbix dengan notifikasi alert Telegram: studi pra-pasca di SDIT Raudhatul As-Salimy Gobah," *Intellect: Indonesian Journal of Learning and Technological Innovation*, vol. 4, no. 1, pp. 107–116, 2025, doi: 10.57255/intellect.v4i1.1372.